

```
if 'extra cheese' in requested_toppings:
    print("Adding extra cheese.")

print("\nFinished making your pizza!")
```

We start with a list containing the requested toppings. The first if statement checks to see whether the person requested mushrooms on their pizza. If so, a message is printed confirming that topping. The test for pepperoni ❶ is another simple if statement, not an elif or else statement, so this test is run regardless of whether the previous test passed or not. The last if statement checks whether extra cheese was requested, regardless of the results from the first two tests. These three independent tests are executed every time this program is run.

Because every condition in this example is evaluated, both mushrooms and extra cheese are added to the pizza:

```
Adding mushrooms.
Adding extra cheese.

Finished making your pizza!
```

This code would not work properly if we used an if-elif-else block, because the code would stop running after only one test passes. Here's what that would look like:

```
requested_toppings = ['mushrooms', 'extra cheese']

if 'mushrooms' in requested_toppings:
    print("Adding mushrooms.")
elif 'pepperoni' in requested_toppings:
    print("Adding pepperoni.")
elif 'extra cheese' in requested_toppings:
    print("Adding extra cheese.")

print("\nFinished making your pizza!")
```

The test for 'mushrooms' is the first test to pass, so mushrooms are added to the pizza. However, the values 'extra cheese' and 'pepperoni' are never checked, because Python doesn't run any tests beyond the first test that passes in an if-elif-else chain. The customer's first topping will be added, but all of their other toppings will be missed:

```
Adding mushrooms.

Finished making your pizza!
```

In summary, if you want only one block of code to run, use an if-elif-else chain. If more than one block of code needs to run, use a series of independent if statements.